Assignment 1

Submission deadline: 09/04/2026
Defence Dates: 07/05/2026 by 11:00 (11 AM)

Link to defence registration: Deep Learning Assignment 1 Defence Schedule.

Part 0: Info

Part 0.1: Questions about the assignment

Questions about the assignment **should be submitted only on the Moodle forum**.
Questions sent by email that are not related to administrative or personal issues (e.g., requesting an extension for justified reasons) will be ignored and will not receive a response.

If you have a question, chances are that others have the same one. Posting on Moodle helps you and your classmates.

Part 0.2: Code Template

We provided a code template you may use, but you are not required to.
The template includes function signatures and notes on common Python methods that you may find helpful. You can extend it and fill in the missing parts.

Part 0.3: Model performance

We don't expect perfect accuracy, but at least one of the submitted models should achieve **at least 45%** accuracy on the test set.

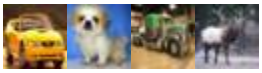
Part 0.4: Introduction

In this assignment, you will train neural networks to recognize objects in images using the CIFAR-10 dataset.

CIFAR-10 contains 50,000 training images and 10,000 test images used to evaluate final model performance.

Each image is an RGB image of size 32×32 pixels.

Here are some examples of images from the dataset:



(This is the actual size of the images)

We'll try to classify the images into 10 different unique classes:
Airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck

Part 0.5: Using GPUs

As mentioned in the lectures and TA sessions, neural networks benefit from GPUs because they can train faster and more efficiently. To support this, you have access to the CS cluster. Make sure you follow the Moodle guides for accessing and using it.
In your code, make sure your tensors and models are actually running on the GPU (device = **cuda**) when you intend to use it.

The assignment is fairly simple and can run on a CPU only. We recommend using a GPU after you develop the full pipeline for training and evaluation for your models.

Part 1: Data loading and exploration (10 points)

1. Load the CIFAR-10 dataset using **torchvision.datasets**:
   https://pytorch.org/vision/main/generated/torchvision.datasets.CIFAR10.html

   Split the dataset into three parts: training, validation, and test.
   - The train set is used to train our neural networks.
   - The validation set is used to evaluate the generalization of the model for different hyperparameter choices.
   - The test set is used for the final evaluation of the model's generalization performance.
2. After loading the data, explore it briefly.
   Please provide **2–3** insightful, **qualitatively distinct** plots/graphs that give useful insights about the dataset, and explain what each plot shows.

Part 2: Basic model training (40 points)

In this part, you will explore different neural-network architectures and implementations for this task.

NOTE: For all experiments in this part, **log and save** the results (e.g., in a list or dictionary) so you can visualize and compare them at the end.

You will implement **2–3** model variants:

1. Implement a “vanilla” MLP that only contains Linear layers and activations of your choice.
    a. Train the MLP on the training set and use the validation set to compare hyperparameters (e.g., number of layers, number of units per layer, learning rate, mini-batch size). Report the experiments you ran and justify your choices.

    b. After you have chosen a set of hyperparameters, generate two line plots: (a) training and validation loss as a function of epoch, and (b) training and validation accuracy as a function of epoch.

    c. Evaluate your “vanilla” MLP on the test set and report the following metrics:
        i. Accuracy
        ii. Loss

2. Next, try to improve performance. Implement a second MLP variant using any additional layers available in **torch.nn** (subject to the restriction below) and/or other techniques covered in class (e.g., L1/L2 regularization, data augmentation, normalization layers, or input normalization).

    Describe what worked and what did not. Report the performance of the best network you trained, as in steps **1a–1c**.

    NOTE: In this assignment you are limited to Linear layers only; implementations with Conv/GRU/LSTM/Attention/Transformer layers will be rejected.

Part 3: CIFAR10-Human (20 points)

In this part, you will use CIFAR-10H for additional evaluation.

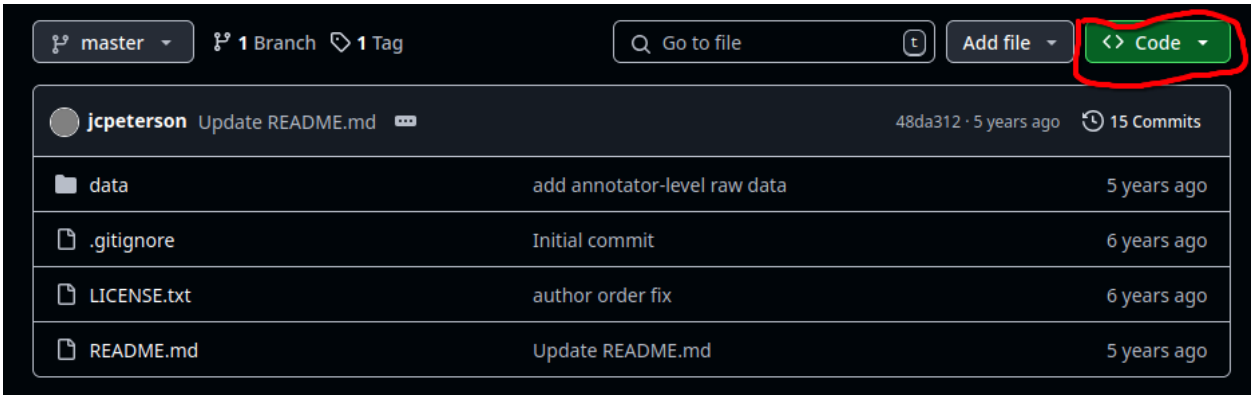
You will also use the model that achieved the best test-set performance in Part 2.
So please make sure you keep that model available (e.g., saved and reloadable) for this part.

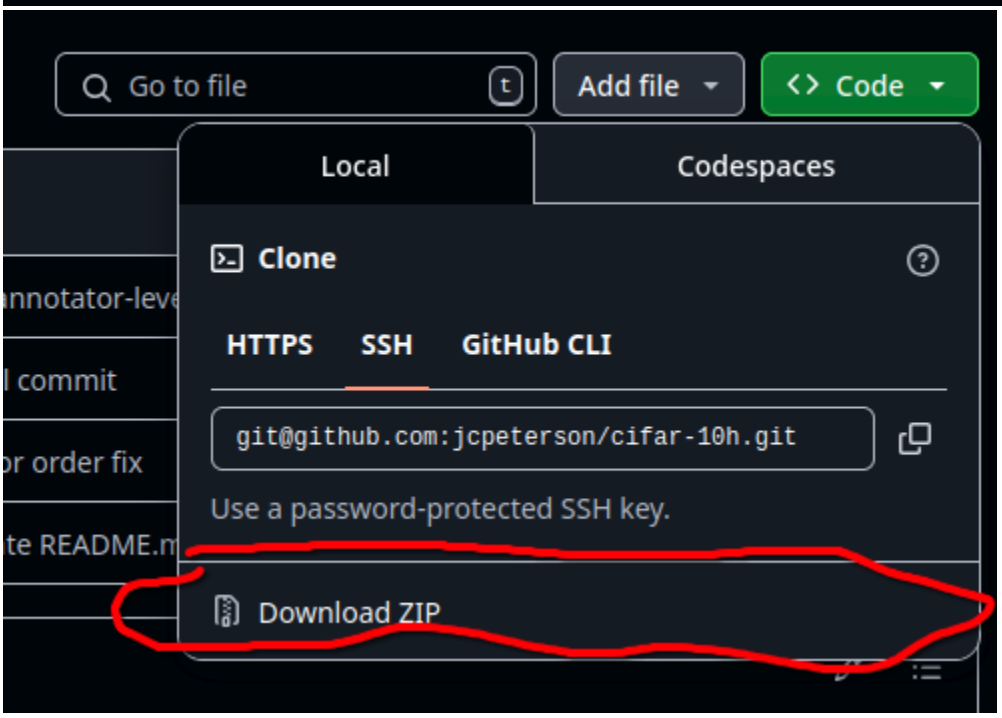
This dataset contains experiments where humans tried to manually label the images in the CIFAR10 test set. It is described in this paper:

Peterson, J. C., Battleday, R. M., Griffiths, T. L., & Russakovsky, O. (2019). Human uncertainty makes classification more robust. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 9617-9626).
(optional reading)

This part will help us understand how neural networks behave compared to human subjects in this task.

Please download the data from: https://github.com/jcpeterson/cifar-10h
If you are unfamiliar with GitHub, press the big green button that says “Code” and then “Download ZIP.”





For the evaluation, you will perform two comparisons between your best model and CIFAR-10H.

1. Please load the “cifar10h-counts.npy” file using the np.load method. This file is a numpy array of size 10,000 by 10 containing votes for each label in the test set by the human subjects.

   For example, if the 5th row contains the following values: [0, 0, 12, 45, 0, 0, 0, 0, 3, 0] That means that 12 subjects thought this image contains a bird, 45 thought it contains a

cat, and three thought it contains a ship.

a. Please extract the index/ID of the label that received the most votes for each test-set image. The result should be a numpy vector or a single list with 10,000 values ranging from 0 to 9.
   For the example above, the index/ID is 3 (arrays start from 0).

b. Find examples/images in the test set where the highest-voted label by the human subjects differs from the actual label in the test set.

   Note: This part should result in 79 values ranging from 0 to 10,000 (which is the image number of the mismatch).

c. Use your best model from part 2 to predict on those 79 images.

d. Extract the highest probability class from the model prediction for those 79 images and compare them to the true labels in the test set.
   Calculate how many images out of 79 the model predicted correctly.

e. Repeat the above step (d), but this time compare it to the human labels.
   Calculate how many images out of 79 the model predicted the same as the most-voted label by humans.

f. Plot a bar plot containing 2 bars, where each bar represents the agreement percentage of the model with either the test set or the human subjects.

g. Which group did the model agree with more: the humans or the true labels? Why do you think that is the case?

2. For this part, we'll load the "cifar10h-probs.npy" file.

   a. Use your model to predict on the entire test set and take the SoftMax-probability value of the model for the true label of all test-set images. This should result in a list of 10,000 values where each value ranges from 0 to 1.0.

      E.g., if the 5th row/image contains the following predicted values: [0.02, 0.1, 0.05, 0, 0, 0.67, 0.16, 0, 0, 0] and the true label for this image is 6, then the value we take is 0.16.

   b. From the cifar10h-probs.npy, take the probability value for the true label as determined by the human subjects; so for the same 5th row/image, we'll take the 6th value from the row.
      This should also result in a list of 10,000 values where each value ranges from 0

to 1.0.

c. Plot a scatter plot where the x-axis is the model probability and the y-axis is the human subjects' probability.

d. Are there any interesting insights you can extract from the graph?

Part 4: BONUS (5 points)

For this part, we'll try to get some insights on the human brain and neural networks.

For this last part, load the "cifar10h-raw.csv" file using **pd.read_csv** from pandas.
In the CSV/DataFrame, each row contains information about each test-set image experiment.

We'll use the reaction time for our analysis.

1. First, compute the average reaction time for each test image by using the groupby and mean methods.

2. Try to compute correlation metrics between the model probabilities for the true label of the test-set image and the reaction time.

   Is there a correlation? In other words, does the model have difficulties with images for which human subjects had high reaction times (took time to label)?

   Are the images with high reaction times also images that the human subjects mislabeled? Did the model label them the same as the humans, or did it predict the correct label regardless?

Submission Instructions:

Please read all the instructions carefully and make sure you meet all the requirements before submitting.

1. Submissions are done in **PAIRS** unless you have special permission.

2. Only **one person** from the pair submits the assignment.

3. Submit a single ZIP file named **ID1_ID2.zip**.

(Replace ID1 and ID2 with the students' actual ID numbers.)

4. The assignment should be a single zip file containing the following:
    a. A PDF file containing graphs and answers to questions
    b. A single **.py** code file that includes all experiments you ran

If the ZIP file contains additional files, points may be deducted.

5. The Python file should be well-organized, follow good coding standards, and be easy to read.